

TouchDesigner Workshop

Creative Tech Studio × ดีดีเอ็กซ์พี · 2 · 3 · 5 มิถุนายน 2026 · 10:30–18:30

ผู้เรียน: 8 (dev-heavy: 4 Fullstack · 2 Frontend · 2 Designer) + 2 PM สังเกตการณ์ · ผู้สอน: Wave Pongruengkiat

Learning Outcomes

1. สร้าง interactive system: input (camera/audio) → process (TD) → output ได้เอง
2. เริ่มโปรเจกต์ TD เองได้ + ประเมิน tech ที่เหมาะกับงานบริษัท
3. ต่อ TD เข้าระบบอื่น (OSC / MQTT / Web / LINE) ได้
4. ทำ mini-project แบบทีม: concept → build → demo
5. รู้วิธีเรียนต่อเอง (docs · community · AI-assisted)

Principles

Project-based ทุก concept จบเป็นไฟล์รันได้ · **Composable** ของ Day 1+2 ประกอบเป็น Day 3 · **Show & Tell daily** ปิดทุกวันด้วยโชว์คนละ 1 ชิ้น · **Dev-first** map TD เข้า mental model เดิม

วิธีอ่าน · แต่ละ session มี

🕒 เวลา + breakdown · 🎯 goal · 1. ความสำคัญ · 2. ประเด็นที่ต้องพูด · 3. เนื้อหา + key scripts · 4. Output · ✂️ เตรียมก่อน · ⌚ run-of-show · 📄 handout · 🔴 กล้องแดง = Gap content แทรกเต็ม

Pedagogy dev vs designer: `teaching-approach.md` · ภาพรวมเวลา: `workshop-schedule.html` · workbook: `handout.html`

⚠️ ไฟล์นี้ = **canonical source** ของเนื้อ session/scripts/run-of-show. **Derived views** ที่ต้องแก้ตามเมื่อเปลี่ยน session (ชื่อ/เวลา/script): `handout.html` + `workshop-schedule.html` + `mock-project-spec.md`

DAY 1 — Foundation & First Interactive Sketch

🎯 ทุกคนมี interactive sketch ที่ใช้ได้จริงบนเครื่องตัวเอง — ปูพื้นแน่นแล้วได้ paint → audio → particle

1.0 Opening: Concept & What We'll Build 10:30–11:00 · 30m Concept 20 / Setup 10

🎯 เห็นภาพ 3 วัน + รู้ว่ากำลังจะสร้างอะไร + เครื่องพร้อมรัน

1. ความสำคัญ

ตั้ง "สัญญา" ของ workshop — ไม่สอน TD เป็นทฤษฎีลอย แต่ build งานจริง (Draw → Scan → Touch) แล้ว fundamental ทุกอย่าง serve ปลายทางนั้น. dev ต้องเห็นปลายทางก่อนถึง engage

2. ประเด็นที่ต้องพูด

- TD = node-based visual programming สำหรับ real-time media (ไม่ใช่ video editor / game engine แต่ยืดแนวคิดทั้งคู่)
- ใช้งานที่ไหน — installation, stage visual, projection mapping, interactive exhibit (โพรเจกต์ 3–4 ชิ้น)
- เชื่อมกับธุรกิจ ดีดีเอ็กซ์พี: "พวกคุณมี service Engagement Tech for Events อยู่แล้ว — TD ทำให้ build installation ลึกขึ้นได้เอง"
- ปลายทาง 3 วัน = mock "Draw → Scan → Touch" · "วันนี้ปูพื้น+sketch, พรุ่งนี้ต่อ input, วันสุดท้าย build+ตั้งโชว์"

3. เนื้อหา + KEY SCRIPTS

ไม่มี script — orientation + verify install. เช็ค: TD เปิดได้ · license non-commercial · webcam/audio เข้า · เปิด mock teaser 30 วิ แล้วปิด

4. OUTPUT

ทุกเครื่องเปิด TD ได้ + เห็น demo ปลายทาง + รู้ว่ากำลังจะไปไหน

🕒 RUN OF SHOW

- | | |
|-------|--|
| 0–3 | เปิด / welcome |
| 3–18 | concept + ตัวอย่างงาน + business framing + teaser mock |
| 18–28 | verify install ที่ละเครื่อง (designer/คนติดยกมือ) |
| 28–30 | segue |

✂ **เตรียม:** mock teaser .toe · slide ตัวอย่างงาน · checklist install

📄 **Handout:** ปก + "What we'll build" diagram + checklist

1.1 TouchDesigner Thinking: Operators & Logic 11:00–12:30 · 90m

Concept 25 / Demo 35 / Hands-on 25 / Share 5

🎯 อ่าน network เป็น · รู้ family ไหนทำอะไร · ต่อค่าข้าม operator ได้ (reference/export)

1. ความสำคัญ

Session ที่ทุกอย่างหลังจากนี้ยืนอยู่บน. ไม่เข้าใจ dataflow + วิธีต่อค่า → interactive ไม่เกิดเลย. จุด map TD เข้า mental model เดิมของ dev (reactive pipeline + data binding)

2. ประเด็นที่ต้องพูด

- TD ไม่ใช่ timeline (อย่าง AE) — เป็น **dataflow** ที่ operator "cook" เมื่อ input เปลี่ยน = reactive
- 5 families = 5 ชนิดข้อมูล: **TOP** ภาพ(GPU) · **CHOP** สัญญาณ/ตัวเลข stream · **SOP** 3D · **DAT** text/table/Python · **COMP** container/UI
- เส้น = ส่ง data; **parameter** ก็ link ข้าม op ได้ โดยไม่ต้องลากเส้น → reference vs export
- segue: "เดี๋ยวเอา concept นี้ไปทำ paint ที่ขยับตามเมาส์จริง"

● GAP 1 WOVEN IN — REFERENCES / EXPORTS (ครู TD ทุกคนยืนยันว่าต้องสอนก่อนเพื่อน)

หัวใจคือ ทิศทางใครเริ่ม: **Reference (Expression) = PULL** ตัวรับไปดึงค่ามาเอง · **Export = PUSH** ตัวส่งยัดค่าเข้าไป

	Reference (Expression)	Export
ทิศ	pull — ตัวรับดึง	push — ตัวส่งยัด
เขียน	พิมพ์ Python ใน parameter	ลาก CHOP channel หั้น parameter
รับจาก	อะไรก็ได้ (param, CHOP, DAT, เวลา)	CHOP channel เท่านั้น
logic/คณิต	ได้ — รวมหลายแหล่งได้	ไม่ได้ — ค่าดิบ
ความเร็ว	ช้ากว่านิด (รัน Python ทุก cook)	เร็วกว่า (ต่อตรง)

เทียบ dev: Reference = computed getter `get radius(){return audio.low*2}` · Export = ต่อ observable stream เข้า prop ตรงๆ `audio.low$.subscribe(v=>circle.radius=v)`

กฎสอน: มี CHOP ไหลๆ ขั้ว param เจๆ → Export · ต้อง logic/รวมค่า/ดึงจากที่ไม่ใช่ CHOP → Reference. parameter modes 4 แบบ: Constant / Expression / Export / Bind

3. เนื้อหา + KEY SCRIPTS

Demo: Circle TOP → Blur → Null , ขั้ว param ดู cook ไหลขวา

```
# Expression (pull) ใน parameter
op('lfo1')['chan1']          # ดึงค่า CHOP channel
op('base1').par.Speed * 2    # custom param มาคูณ
absTime.seconds              # เวลา (วินาที)
```

Export (push): ลาก channel จาก LFO CHOP → drop หั้น parameter radius (เปลี่ยนเป็นสีเขียวเทา = ถูก export คม)

shortcut: Tab สร้าง op · u ขึ้น · i เข้า COMP · h home · middle-click = info. **DRY:** วาง Null เป็น anchor ก่อน reference (แก้ network ไม่พัง = abstraction layer)

4. OUTPUT

network ที่ LFO ขั้ว param ผ่าน export + อีกตัวอ่านผ่าน expression · ผู้เรียนอ่าน network ออก นอกทิศ data ได้ · เชฟ .toe ตัวแรก

🕒 RUN OF SHOW

- 0–25 mini-lecture: families + dataflow + ref/export
- 25–60 live demo (สร้าง network ขั้วๆ, อธิบาย ref vs export)
- 60–85 hands-on: LFO→export→Circle + expression อ่านค่า
- 85–90 ใครติดยกมือ + segue

✂ **เตรียม:** demo .toe เปลา · slide "5 families" · cheat sheet shortcut

📄 **Handout:** Operators & parameter modes + ตาราง ref vs export + ช่องจด

1.2 Tutorial 1: Visual Interactive (Mouse → Feedback) 13:30–14:45 · 75m

Concept 10 / Demo 25 / Hands-on 35 / Share 5

🎨 paint app ที่เมาส์วาดแล้วเส้นค้าง — เข้าใจ feedback loop + ใช้ ref/export จริง

1. ความสำคัญ

Sketch แรกที่ทำเอง — ใช้ ref/export จาก 1.1 ทันที. **Feedback loop** จะกลับมาใช้ตอน particle trail + drawing accumulation. ใช้เมาส์ (ยังไม่แตะกลอง) เพื่อโฟกัส TD logic ก่อน

2. ประเด็นที่ต้องพูด

- Feedback loop = output เฟรมก่อนป้อนกลับเข้าเฟรมนี้ = "ความจำ" ของภาพ (เหมือน state ที่ accumulate)
- ทำไม Feedback TOP ต้องคู่ Composite/Over — วาดของใหม่ทับของเก่าที่จาง
- decay = คุณ feedback ด้วยค่า <1 ทุกเฟรม · segue: "ลองเปลี่ยน input จากเมาส์เป็นเสียง"

3. เนื้อหา + KEY SCRIPTS

Chain: Mouse In CHOP → Circle (brush) → Composite(over) ← Feedback TOP → Level(×0.95 fade) → out

```
op('mousein1')['tx'] # brush Translate X
op('mousein1')['ty'] # brush Translate Y
```

decay ที่ Level TOP ~0.95 → เส้นค่อยจาง · brush size/สี = designer เล่นตรงนี้

4. OUTPUT

paint sketch: ลากเมาส์ = เส้นเรืองแสงค่อยจาง · เข้าใจ feedback loop ที่จะ reuse · เชฟ .toe

🕒 RUN OF SHOW

- | | |
|-------|--|
| 0–10 | concept feedback |
| 10–35 | live demo (สร้าง paint ทีละ node) |
| 35–70 | hands-on (ทำ + ปรับ brush/decay; designer จูนสี) |
| 70–75 | quick share |

✂ **เตรียม:** demo paint .toe · slide "feedback = state accumulation"

📄 **Handout:** node diagram paint + ช้องจด

1.3 Tutorial 2: Audio Interactive (Mic → Visual) 14:45–15:30 · 45m

Concept 8 / Demo 17 / Hands-on 15 / Share 5

🎧 เอาเสียงจริงขับภาพ — CHOP เป็น input stream + audio analysis

1. ความสำคัญ

Input ตัวที่สอง (เสียง) — ตอกย้ำ "อะไรก็เป็น CHOP ได้ แล้ว export ไปขับ param". audio-reactive = hello-world ของ TD, เร็วและ rewarding

2. ประเด็นที่ต้องพูด

- เสียง → ตัวเลข: Audio Device In → level (ดังแค่ไหน) หรือ spectrum (ความถี่ไหนแรง)
- level = 1 ค่า · spectrum = หลายค่า (bass/mid/treble) · "pattern เดียวกับเมื่อก็แค่เปลี่ยนต้นทาง"

3. เนื้อหา + KEY SCRIPTS

Audio Device In → Analyze(RMS) → Lag → export · spectrum: Audio Spectrum CHOP · normalize ด้วย Math CHOP (range 0..1) ก่อนใช้

4. OUTPUT

ภาพเดินตามเสียง (ตบมือ/พูด → visual ขยับ) · เห็น pattern input→analyze→export→visual ซ้ำ · เชฟ .toe

🕒 RUN OF SHOW

- | | |
|-------|-------------------------------------|
| 0–8 | concept level/spectrum |
| 8–25 | demo |
| 25–40 | hands-on (เสียง → ขยับ paint/วงกลม) |
| 40–45 | share |

✂ **เตรียม:** demo audio-reactive .toe · เช็ค mic

📄 **Handout:** chain audio + Analyze vs Spectrum

1.4 Tutorial 3: First Particle Sketch 15:45–17:45 · 120m Concept 20 / Demo 40 / Hands-on 50 / Share 10

🎯 particle ที่ react ได้ + เข้าใจ instancing และ 3D render pipeline

1. ความสำคัญ

ขั้นใหญ่สุดของ Day 1 และเป็น หัวใจของ **mock**. เทคนิคนี้ Day 3 จะ reuse (เปลี่ยนต้นทางจาก noise → รูปวาด). มีเวลา
มากที่สุด ลงให้แน่น

2. ประเด็นที่ต้องพูด

- particle ใน TD ไม่ใช่ของวิเศษ = "วาด mesh ขึ้นเดียวซ้ำหลายที" = instancing
- ของ 3D ต้องมี render pipeline · ต้นทางตำแหน่งมาจาก CHOP/SOP — วันนี้ noise, วันสุดท้ายเป็นฟิกเชลรูปวาด
- segue: "พรุ่งนี้เอามือมาดันฝูง particle นี้"

● GAP 3 WOVEN IN — INSTANCING & 3D RENDER PIPELINE

ปัญหา: อยากวาดทรงกลม 50,000 ลูก — สร้างแยกกัน = คายสนิท. **Instancing** = วาดของชิ้นเดียวซ้ำหลายที ในการวาดครั้งเดียว: เก็บ mesh อันเดียว + ยื่น list ตำแหน่งให้ GPU → GPU บั้มในจังหวะเดียว. 50,000 ลูก ≈ ต้นทุนเกือบเท่าลูกเดียว

กลไก: mesh ต้นแบบ 1 อัน + ตารางต่อ instance (tx ty tz, หมุน, ขนาด, สี). **1 sample ใน CHOP = 1 instance**

เทียบ dev: instancing = for-loop บน GPU = `InstancedMesh` (Three.js) / `gl.drawArraysInstanced` — frontend คนเคย WebGL เก๋าคำนึงทันที. ไม่มี instancing = เรียก `draw()` 50,000 ครั้ง (CPU คอขวด); มี = 1 draw call

3D render pipeline ขั้นต่ำ: `Geometry COMP + Camera COMP + Light COMP → Render TOP → ภาพ`

โยงงานเรา: Day 3 จะ sample พิกเซลรูปวาด → แต่ละจุดเป็นตำแหน่ง+สีของ instance → รูปกลายเป็นฝูง particle.

Method fork: 3D instanced (ทำตามนี้) ต้องมี render pipeline · ถ้าเลือก 2D GPU particle TOP จะไม่ต้องแตะ 3D —

Wave เลือก method ตัดสินตรงนี้

3. เนื้อหา + KEY SCRIPTS

pipeline: `Geometry COMP` (Sphere เล็ก) + `Camera` + `Light` → `Render TOP` · เปิด **Instancing** บน `Geo COMP` ที่ source ไป CHOP

ตำแหน่ง: `Grid SOP (50×50) → SOP to CHOP` (P0 P1 P2 = tx ty tz) หรือ `Noise CHOP`

```
# ขยับ instance ตามเวลา+เสียง (CHOP Math/Expression)
me.inputVal + op('audio_level')['chan1'] * 0.5
```

per-instance scale/count ขยับด้วย audio (export จาก 1.3) · เก็กรัน GLSL สำหรับคนไว (optional)

4. OUTPUT

ฝูง particle (instanced sphere) ขยับ/เปลี่ยนขนาดตามเสียง · เข้าใจ instancing + render pipeline = พร้อม reuse Day 3 · เชฟ .toe

🕒 RUN OF SHOW

- 0–20 concept instancing + 3D pipeline
- 20–60 live demo (Grid→instance→render ทีละขั้น)
- 60–110 hands-on (instance + ขยับเสียง; คนไวลอง GLSL)
- 110–120 share

✂ **เตรียม:** demo particle .toe · slide "instancing = GPU loop" + เทียบ `InstancedMesh` · fallback file

📄 **Handout:** diagram render pipeline + instancing source

1.5 Show & Tell — Day 1 17:45–18:30 · 45m Demo 30 / Discussion 15

🎯 ทุกคนโชว์ 1 sketch + เห็นงานเพื่อน + Wave จับ level ทิมไว้ปรับ Day 2

1. ความสำคัญ

หลัก Show & Tell daily — บังคับให้ทุกคนมี "ของเสร็จ" 1 ชิ้น. Wave อ่าน level จริงเพื่อปรับ pace Day 2 (ใคร peer helper, ใครต้องประคอง)

2. ประเด็นที่ต้องพูด

- "โชว์คนละชิ้น ไม่ต้องเพอร์เฟกต์ — เล่าว่าทำอะไร ติดตรงไหน"
- ชี้ pattern ร่วม: ทุกชิ้น = input→process→output · preview Day 2

3 + 4. เนื้อหา / OUTPUT

presentation + feedback. Wave จดว่าใครแข็ง (peer helper Day 3) ใครต้องประคอง · ทุกคนมี ≥1 sketch โชว์ได้

🕒 RUN OF SHOW

0–3	setup
3–33	โชว์ทีละคน ~3 นาที
33–43	Wave สรุป pattern + preview Day 2
43–45	ปิดวัน

📄 **Handout:** ช่อง reflection "วันนี้ทำอะไรได้ / ติดอะไร / พรุ่งนี้อยากลองอะไร"

DAY 2 — Input Systems & Integration

🎯 ต่อ input จริง (มือ + กล้อง) เข้า TD · เก็บข้อมูลลงไฟล์ · เห็นว่า TD ต่อเข้าระบบอื่นได้

2.0 Recap & Goal 10:30–11:00 · 30m Recap 20 / Setup 10

🎯 เชื่อม Day 1 → Day 2 · ทุกคนพร้อมเครื่อง + เข้าใจว่าวันนี้สร้าง "ระบบ input"

1. ความสำคัญ

ดึง momentum + ย้ำ mental model ก่อนเพิ่มความซับซ้อน · dev ที่หลุดเมื่อวานได้ตามทัน

2. ประเด็นที่ต้องพูด

- recap: ทุก sketch = input(เมาส์/เสียง)→export→visual
- วันนี้เปลี่ยน input เป็นของจริง: มือ(MediaPipe) + กล้อง+กระดาษ(ArUco) + เก็บไฟล์ · ปิดด้วย "TD ต่อระบบบริษัทยังไง"

3 + 4. เนื้อหา / OUTPUT

เปิดไฟล์เมื่อวาน · เช็ก plugin MediaPipe + webcam · ทุกเครื่องพร้อม + เห็น roadmap

🕒 RUN OF SHOW

- 0-3 เปิด
- 3-20 recap + ถาม-ตอบค้างคา
- 20-30 verify plugin/webcam + segue

✂ **เตรียม:** เช็ค mediapipe-touchdesigner plugin ลงครบทุกเครื่อง (สำคัญ — ทำก่อนเริ่ม)

2.1 Body & Hand Tracking (MediaPipe → CHOP) 11:00-12:30 · 90m

Concept 15 / Demo 30 / Hands-on 40 / Share 5

🎯 **ดึงตำแหน่งมือ/ตัวจากกล้องเป็น CHOP แล้วขับภาพ**

1. ความสำคัญ

Input ตัวเอกของ "Touch" ใน mock. ใช้ **mediapipe-touchdesigner plugin** (Torin Blankensmith) รัน MediaPipe ใน TD ผ่าน embedded Chromium — ไม่ต้องเขียน Python. ออกมาเป็น CHOP = ต่อ pattern เดิมได้ทันที. **DDEXP ทำ AI/ML อยู่แล้ว → เข้าง่าย**

2. ประเด็นที่ต้องพูด

- MediaPipe = ML model ของ Google หา landmark มือ/ตัว/หน้า จากภาพกล้อง
- plugin → output เป็น CHOP channel (ปลายนิ้ว tx/ty/tz) = "input stream อีกตัว เหมือนเมาส์/เสียง"
- ค่าเป็น normalized 0..1 ต้อง map · ต้อง Lag/Filter ลด jitter

3. เนื้อหา + KEY SCRIPTS

```
op('handtrack')['index_tip:tx'] - 0.5 # map 0..1 → -0.5..0.5
0.5 - op('handtrack')['index_tip:ty'] # flip Y
# Lag/Filter CHOP ลด jitter · Slope CHOP = ความเร็วมือ (แรงผลักดันหลัง)
```

ชื่อ channel ขึ้นกับ version ของ plugin — เช็ค viewer

4. OUTPUT

มือขยับหน้ากล้อง → brush/particle ตาม · เชื่อม MediaPipe→CHOP→visual ได้ · เชฟ .toe

🕒 RUN OF SHOW

- 0-15 concept MediaPipe + normalized coords
- 15-45 demo (วาง plugin → map นิ้ว → ขับ brush)
- 45-85 hands-on (มือขับ particle/paint)
- 85-90 share

✂ **เตรียม:** plugin ลงครบ · demo .toe · lighting ห้องพอให้ กล้องเห็นมือ

📄 **Handout:** chain MediaPipe→CHOP + coordinate mapping

2.2 Computer Vision (Webcam + ArUco → Homography) 13:30–14:45 · 75m

Concept 15 / Demo 30 / Hands-on 25 / Share 5

 scan รูปจากกระดาษให้ตรง (ตัด perspective) ด้วย ArUco + OpenCV

1. ความสำคัญ

ขึ้น **custom** สุด นอกตำรา TD (ไม่มีใน course ไหน) = ส่วนเสียงสุดของ mock → allocate mentoring เยอะ. จุดที่ dev ได้เขียน Python จริงใน Script TOP — flatter สาย code

2. ประเด็นที่ต้องพูด

- ปัญหา: กล้องมองกระดาษเฉียง → ภาพบิด. ต้อง "ดึงให้ตรง" = homography/perspective warp
- ArUco = marker 4 อันมุมกระดาษ ใช้หาพิกัด 4 มุม → คำนวณ transform → warp เป็นสี่เหลี่ยมตรง
- Script TOP = เขียน Python (OpenCV) ได้ "นี่คือที่ที่พวกคุณเขียนโค้ดเต็มที่" · caveat: ลง `opencv-contrib-python`

3. เนื้อหา + KEY SCRIPTS (ILLUSTRATIVE — ปรับตาม OPENCV VERSION)

Video Device In TOP → Script TOP (ArUco+warp) → out

```
import cv2, numpy as np

def onCook(scriptOp):
    src = scriptOp.inputs[0].numpyArray(delImageData=True)
    img = np.flipud((src[:, :, :3]*255).astype(np.uint8)) # TD ภาพกลับหัว
    gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
    adict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)
    detector = cv2.aruco.ArucoDetector(adict, cv2.aruco.DetectorParameters()) # OpenCV >= 4.7
    corners, ids, _ = detector.detectMarkers(gray)
    if ids is not None and len(ids) >= 4:
        pts = order_by_id(corners, ids) # helper เขียนเอง
        W = H = 1024
        M = cv2.getPerspectiveTransform(pts, np.float32([[0,0],[W,0],[W,H],[0,H]]))
        warped = cv2.warpPerspective(img, M, (W,H))
        out = np.dstack([warped, np.full((H,W),255,np.uint8)]).astype(np.float32)/255
        scriptOp.copyNumpyArray(np.flipud(out))
    return
```

4. OUTPUT

กล้องเล็งกระดาษมี marker → เห็นภาพถูกดึงให้ตรงใน TD (ยังไม่ save) · เชฟ .toe

RUN OF SHOW

- 0–15 concept homography + ArUco
- 15–45 demo (Script TOP + warp, อธิบายโค้ด)
- 45–70 hands-on (dev เล่นโค้ด · designer ถือกระดาษ/จัดกล้อง)
- 70–75 share

✂ **เตรียม:** ลง opencv-contrib ทุกเครื่อง · พิมพ์ ArUco template · demo .toe · order_by_id helper

📄 **Handout:** flow scan + โค้ด ArUco เดิม (copy ได้) + note ลง opencv-contrib

2.3 Data Capture (Save → File → Auto-load) 14:45–15:30 · 45m

Concept 10 / Demo 20 / Hands-on 10 / Share 5

🎯 save รูป scan ลงไฟล์ + TD โหลดไฟล์ใหม่กลับเองอัตโนมัติ

1. ความสำคัญ

ปิด loop kiosk: วาด→scan→เก็บ→โหลดกลับ. file I/O + folder watch = pattern ที่ใช้กับงาน data-driven ของบริษัทได้ (dev relevant)

2. ประเด็นที่ต้องพูด

- save ภาพ: Movie File Out TOP (snapshot) หรือ cv2.imwrite · ตั้งชื่อด้วย timestamp
- folder watch Folder DAT + auto-load DAT Execute → "event-driven file pipeline เหมือน file watcher ที่พวกคุณเขียน"

3. เนื้อหา + KEY SCRIPTS

```
op('warped').save(f'scans/draw_{int(absTime.seconds)}.png')

# Folder DAT → DAT Execute (onTableChange)
def onTableChange(dat):
    latest = dat[dat.numRows-1, 'name'].val
    op('moviefilein1').par.file = f'scans/{latest}'
    return
```

4. OUTPUT

กดบันทึก → ไฟล์ PNG เกิด → TD ตั้งโหลดภาพล่าสุดเอง · เชฟ .toe

🕒 RUN OF SHOW

- | | |
|-------|--------------------------------------|
| 0–10 | concept file pipeline |
| 10–30 | demo (save + Folder DAT + auto-load) |
| 30–40 | hands-on |
| 40–45 | share |

✂ **เตรียม:** demo .toe save+watch · โฟลเดอร์ scans/

📄 **Handout:** chain save/watch + โค้ด (copy ได้)

2.4 Web → Interactive (TD as Server) 15:45–17:15 · 90m

 เอา web stack ของ DDEXP มาใช้ในงาน interactive — TD เป็น server, มือถือควบคุม installation ผ่านเว็บ

1. ความสำคัญ

Session ที่ทำให้ **workshop** เป็นของ **DDEXP** เต็มตัว. จุดแข็ง #1 ของเขา = web/frontend/LINE — แทนที่จะทิ้งแล้วเรียน TD ใหม่หมด, อันนี้ เอา **superpower** เดิมมาใช้ใน **installation**. pattern "มือถือคนดู → คุมงานผ่านเว็บ" คือหัวใจของ Engagement Tech for Events ที่เขาขายอยู่แล้ว

2. ประเด็นที่ต้องพูด

- **TD เป็น server ได้** — HTTP + WebSocket ในตัว (Web Server DAT)
- 4 ตัว: Web Render TOP (เว็บ→texture) · Web Server DAT (TD host หน้าเว็บ+WS) · WebSocket DAT (ต่อ backend) · Web Client DAT (เรียก REST/LINE API)
- เทียบ dev: Web Server DAT = Express + ws ผังใน TD · Web Render TOP = iframe/Electron ที่ output เป็น texture · WebSocket DAT = ws client · Web Client DAT = fetch()
- "OSC / MIDI / DMX / Serial ก็ pattern เดียวกัน (DAT/CHOP รับ-ส่ง) — ไม่ลงลึกวันนี้" (ลดจากเดิม)

★ WEB ↔ TD — 4 ตัวหลัก + PATTERN เด็ด

Operator	ทำอะไร	เทียบ dev
Web Render TOP	รันเว็บ/Three.js/canvas สด → texture ใน TD	iframe / Electron window
Web Server DAT	TD เป็น HTTP+WebSocket server — เซิร์ฟหน้าเว็บ + รับ message	Express + ws
WebSocket DAT	TD เป็น client ต่อ WS server ภายนอก (backend, Mosquitto ws://9001)	new WebSocket()
Web Client DAT	ยิง HTTP GET/POST → REST / LINE API / CRM	fetch() / axios

Pattern เด็ด — phone-as-controller: มือถือเปิดหน้าที่ TD เซิร์ฟ → แตะ/ลาก → WebSocket → TD ขั้ว particle/scene → broadcast กลับทุกเครื่อง. คนดูหลายคนเล่นพร้อมกันผ่านมือถือ ไม่ต้องแจก controller = engagement tech for events

3. เนื้อหา + KEY SCRIPTS

```
# Web Server DAT callbacks – TD เป็น server
def onHTTPRequest(webServerDAT, request, response):
    response['statusCode'] = 200
    response['data'] = op('controlpage').text # HTML ใน Text DAT
    return response

def onWebSocketReceiveText(webServerDAT, client, data):
    import json
    msg = json.loads(data) # จากมือถือ
    op('control').par.value0 = msg.get('push', 0) # ขั้ว parameter
    return
# broadcast กลับทุก client: webServerDAT.webSocketSendText(text)
```

```
# หน้าเว็บที่เสิร์ฟ (controlpage Text DAT)
<script>
const ws = new WebSocket('ws://' + location.host);
document.body.onclick = e => ws.send(JSON.stringify({push: 1.0}));
</script>
```

4. OUTPUT

มือถือเปิดเบราว์เซอร์ → ตะ → particle ใน TD ขยับ (phone คม installation จริง) · dev เห็นว่าเอา web stack ตัวเองมาต่อ TD ได้ · เซฟ .toe + control-page HTML

🕒 RUN OF SHOW

- 0–20 concept: Web Render / Web Server / WebSocket / Web Client + pattern phone-controller
- 20–60 demo: Web Server DAT เสิร์ฟหน้าเว็บ → มือถือแตะ → ขั้ว particle (+ Web Render TOP เอาเว็บมาเป็น layer)
- 60–85 hands-on (มือถือตัวเองคุม visual ผ่าน WS; ต่อ backend/LINE ถ้าไหว)
- 85–90 share

✂ **เตรียม:** demo .toe (Web Server DAT + particle) + control-page HTML สำเร็จ · เครื่องกับมือถืออยู่ WiFi เดียวกัน · เช็ก IP/firewall port

📄 **Handout:** 4 web operators + Web Server DAT callbacks + control-page HTML (copy ได้)

2.5 Show & Tell — Day 2 17:15–18:30 · 75m Demo 40 / Discussion 20 / Brief 15

🎯 โขว์ระบบ input + capture + เตรียมหัวเข้า Day 3 (มอง mock เป็นขั้นเดียว)

1. ความสำคัญ

จุดบรรจบ — มีชิ้นส่วน (มือ, scan, save, integrate) ครบ. ทำให้เห็นว่า "พรั่งนี้แค่เอามาต่อกัน" ไม่ใช่เริ่มใหม่ (composable)

2. ประเด็นที่ต้องพูด

- โขว์ระบบ input ของแต่ละคน · ขั้วทุกชั้น = บล็อกของ mock
- brief mock Day 3 ล่วงหน้า ให้ไปคิดข้ามคืน + แบ่งทีม

3 + 4. เนื้อหา / OUTPUT

demo + planning · ทุกคนมีระบบ input ≥ 1 ที่ทำงาน + เห็นภาพ Day 3 = ประกอบ ไม่ใช่เริ่มใหม่

🕒 RUN OF SHOW

- 0-3 setup
- 3-43 โชว์ทีละคน
- 43-58 discussion (ขึ้นส่วนต่อกันยังไง)
- 58-73 brief mock Day 3 + แบ่งทีม
- 73-75 ปิดวัน

📄 **Handout:** reflection + "ขึ้นส่วนของฉันที่จะเอาไปต่อ Day 3"

DAY 3 — Ship Your Mini Project

🌀 ประกอบเป็น mock จริง → ทำ UI → คิดเรื่องการตั้งโชว์ → ติดตั้งจริง → นำเสนอ

3.0 Brief & Recap: the Mock 10:30-11:00 · 30m Brief 20 / Team setup 10

🌀 ทุกทีมเข้าใจ spec mock + แบ่งงานในทีม

1. ความสำคัญ

ตั้งโจทย์วันสุดท้ายให้ชัด ป้องกันหลงทาง · ย้ำว่าใช้ของที่ทามาแล้ว 2 วัน

2. ประเด็นที่ต้องพูด

- spec: **Draw** → **Scan** → **Particle** → **Hand-interact** · ทุกทีมทำตัวเดียวกัน (baseline) · คนไวต่อ stretch (body pose)
- บทบาท: dev คม logic/integration · designer คมภาพ/template/นำเสนอ · timeline: build เข้า · UI+install ปลาย · present เย็น

3 + 4. เนื้อหา / OUTPUT

review ขึ้นส่วนที่จะต่อ: instancing(1.4) + scan(2.2) + save(2.3) + hand(2.1) · ทุกทีมรู้ว่าทำอะไร + ใครทำส่วนไหน

🕒 RUN OF SHOW

- 0-20 brief spec + demo เป้าหมาย
- 20-30 แบ่งทีม/บทบาท + segue

✂ **เตรียม:** spec mock 1 หน้า · diagram ขึ้นส่วน→mock

📄 **Handout:** spec + diagram pipeline + ช่องวางแผนทีม

3.1 Build Time: Compose & Live Mentoring 11:00–12:30 · 90m Concept 15 / Build 70 / Share 5

🎯 ต่อชิ้นส่วนเป็น mock ที่ทำงาน + เรียน modularity (COMP/.tox)

1. ความสำคัญ

หัวใจ Day 3. drawing→particle ใช้ instancing จาก 1.4 (เปลี่ยน source เป็นรูปวาด). team dev บริษัทคิดเรื่อง architecture อยู่แล้ว — สอนห่อ COMP = กัน spaghetti + reuse ข้ามทีม

2. ประเด็นที่ต้องพูด

- ประกอบ: รูป scan(2.2/2.3) → source ของ instance (แทน noise ใน 1.4) → มือ(2.1) ดัน particle
- เปลี่ยนรูป→particle: sample พิกเซลที่มีสี → ตำแหน่ง+สีของแต่ละ instance · "อย่าสร้าง network ก้อนเดียวยักษ์ — ห่อเป็นโมดูล"

● GAP 2 WOVEN IN — COMPS / .TOX / MODULARITY ("BUILD LIKE ENGINEERS" = วัฒนธรรม ดีดีเล็กซ์พี)

COMP (Component) = กล่องที่ห่อ network ทั้งก้อน. สร้างครั้งเดียว → ห่อ → เปิด custom parameter เป็น "หน้าบ้าน/API" → reuse ได้

TD	โลก code
COMP	class / module / component
Custom parameter	constructor args / props
.tox file	npm package / importable module
ลาก COMP หลายตัว	instantiate objects · DRY

ตัวอย่าง: particle system เป็น COMP ที่มี param color/count/speed → ทีม A และ B ใช้ COMP เดียวกัน ปรับ param ต่างกัน ไม่ต้อง copy-paste network

```
parent().par.Pushforce      # อ่าน custom param ของ COMP แม่ จากข้างใน
# ห่อ: เลือก network → RMB → Collapse Selected → COMP
# Custom Parameter (Component Editor) → ตั้ง param
# export: RMB COMP → Save Component .tox → ลากกลับเข้าโปรเจกต์ไหนก็ได้
```

3. เนื้อหา + KEY SCRIPTS

รูป→instance: Movie File In (รูป scan) → TOP to CHOP (sample พิกเซลเป็นตำแหน่ง/สี) → instance source · มือดัน: hand velocity (Slope จาก 2.1) → offset ตำแหน่ง instance

4. OUTPUT

mock: scan รูป → particle ตามรูป → บัดมือ particle กระจาย · อย่างน้อย 1 ชิ้นห่อเป็น .tox มี custom param · เชฟ .toe + .tox

🕒 RUN OF SHOW

- 0–15 concept compose + COMP/.tox
- 15–85 build (Wave + peer เดินช่วยทีละทิม — เน้น ArUco/instance ที่เสี่ยง)
- 85–90 quick share สถานะ

✂ **เตรียม:** demo mock เต็ม · ชิ้นส่วน .tox สำรอง (scan.tox, particles.tox) · peer helper จาก Day 1

📄 **Handout:** diagram compose + วิธีห่อ COMP/.tox + custom param

3.2 TouchDesigner UI Essentials 13:30–14:30 · 60m Concept 12 / Demo 23 / Hands-on 20 / Share 5

🎯 control panel คมงานหน้างาน + presets — เบาๆ พอใช้ได้ (dev มี UI framework เองอยู่แล้ว)

1. ความสำคัญ

งาน install ต้องมีหน้าคุม (เปิด/รีเซ็ต/ปรับค่า) โดยไม่แตะ network. สอนเบตามที่ research แนะนำ — dev เก่ง UI อยู่แล้ว

2. ประเด็นที่ต้องพูด

- คนคุมงานไม่ควรต้องงม network สด · TD UI = COMP (Container/Button/Slider) + อ่านค่าเข้า Panel CHOP
- custom parameter (จาก 3.1) = วิธีเร็วสุด · presets = Storage / parameter snapshot · "ไม่ต้องสวย แค่ใช้ได้ + reset ได้"

3. เนื้อหา + KEY SCRIPTS

```
# ปุ่ม reset (Button callback)
def onClick(panelValue):
    op('feedback1').par.resetpulse.pulse() # ล้าง feedback/particle
    return
```

panel: Container + Button/Slider → Panel CHOP อ่านค่า → export เข้า param · presets ผ่าน Storage

4. OUTPUT

control panel เล็ก: reset + 2–3 slider คมงาน · เชฟ .toe

🕒 RUN OF SHOW

- 0–12 concept UI/custom param
- 12–35 demo (custom param + reset + slider)
- 35–55 hands-on (ทำหน้าคุม mock; designer จัด layout)
- 55–60 share

✂ **เตรียม:** demo UI .toe · template container

📄 **Handout:** วิธีเปิด custom param + ปุ่ม reset code

3.3 Installation Thinking: Experience & Placemaking 14:30–15:30 · 60m

💡 คิดงานเป็น "ประสบการณ์ในพื้นที่" ไม่ใช่แค่ของบนจอ — วางแผน install จริง

1. ความสำคัญ

ส่วน **value-add** ที่ **Wave (cultural futurist)** เติม — ไม่มีใน technical TD course. เปลี่ยน mindset จาก "ทำ effect" เป็น "ออกแบบประสบการณ์คน". **designer 2** คนเป็น **service designer** มีอาชีพ → **ดึงดูดเชิงเขาเต็มที่**

2. ประเด็นที่ต้องพูด

- งาน interactive อยู่ใน "พื้นที่" ไม่ใช่แค่จอ — คนเดินเข้ายังงี้, รู้ได้ไงว่าเล่นได้, เล่นเสร็จแล้วงี้
- วงจร: **attract** → **invite** → **engage** → **release**
- placemaking — งานเปลี่ยนความหมายพื้นที่ · sightline, แสง, เสียง, การจราจรคน
- idle/attract state · ความทน (รันทั้งวัน, reset ง่าย, ไม่พังถ้าคนทำแปลกๆ) · "คิด UX ของพื้นที่จริงเหมือน UX ของ app"

3 + 4. เนื้อหา / OUTPUT

discussion + planning · โจทย์: ทีมวาดผังการตั้ง mock (กล่อง/จอ/projector/คนยืน/idle state) · ได้แผนตั้งงาน + mindset experience-first

🕒 RUN OF SHOW

- 0–20 mini-lecture experience/placemaking + ตัวอย่างงาน install
- 20–45 discussion + ทีมวาดผัง install
- 45–60 แต่ละทีมเล่าผัง + segue install จริง

✂ **เตรียม:** slide attract→release + ตัวอย่างงาน install · กระดาษ/ปากกาวาดผัง

📄 **Handout:** วงจร attract→release + checklist install + ชื่องานผัง

3.4 Physical Prototyping & Live Install 15:45–17:30 · 105m

Build 30 / Install 45 / Optimize 20 / Buffer 10

💡 ตั้ง mock เป็น installation จริง (cardboard + projector) + จูน performance ให้รันลื่น

1. ความสำคัญ

เอา digital ออกมาสู่ physical — cardboard scan station + ยิง projector จริง. งานรันสดหน้าคน FPS ห้ามตก

2. ประเด็นที่ต้องพูด

- ประกอบ cardboard scan station (ฐานกล่อง + กรอบกระดาษ) — craft + tech
- ยิง projector: resolution, aspect ratio, fullscreen (Window COMP) · reset/idle จากแผน 3.3

● GAP 4 WOVEN IN — OPTIMIZATION (กัน FPS ตกหน้างาน)

งาน install รันสด → ห่วง **frame rate**. concept หลัก:

- **Cook** — operator คำนวณใหม่เมื่อ input เปลี่ยน. อะไร cook ทุก frame โดยไม่จำเป็น = กิน FPS
- **Resolution** — TOP ที่ 4K แพงกว่า 1080p 4 เท่า · ใช้ res ให้พอดี
- **GPU vs CPU** — งานอยู่บน GPU (TOPs) เร็ว · บาง CHOP/SOP เป็น CPU คอขวดได้

เทียบ **dev**: profiling + เลี่ยงคำนวณซ้ำ (memoization) + เก็บ hot path บน GPU + คำนวณ frame budget (16.6ms ต่อ 60fps)

```
op('render1').cookTime          # cook time (ms)
# Perform Mode = F1 (fullscreen) · Performance Monitor = Alt+Y · Probe (palette)
```

จน: ลด res ที่ไม่จำเป็น · Null + selective cook · Cache/bypass สาขาไม่ใช่ · instance แทน copy (มีแล้ว) · idle state = timer ไม่มี input → กลับ attract loop

3. เนื้อหา + KEY SCRIPTS

fullscreen: Window COMP → ตั้ง projector → Perform Mode (F1)

4. OUTPUT

mock ตั้งเป็น installation จริง ยิง projector เล่นได้ · รันสั้น (รู้วิธีหา/แก้จุดอืด) · มี idle/reset

🕒 RUN OF SHOW

- | | |
|--------|---|
| 0–30 | ประกอบ station + ยิง projector |
| 30–75 | install + จูนหน้างาน (Wave/peer ช่วย) |
| 75–95 | optimization pass (หา/แก้จุดอืด + idle/reset) |
| 95–105 | buffer/แก้ปัญหา |

✂ **เตรียม:** cardboard + เครื่องมือ · projector (อันเล็ก office — verify) · webcam + extender · demo Perform/Probe

📄 **Handout:** checklist install + Window COMP/Perform + วิธีดู cook time + tips optimize

3.5 Final Presentation 17:30–18:30 · 60m Demo 35 / Retro 15 / How-to-continue 10

🎯 แต่ละทีมโชว์งานที่ตั้งจริง + retro + วิธีเรียนรู้ต่อเนื่อง

1. ความสำคัญ

ปิด workshop ด้วยของจริงที่ "ship" แล้ว (เป้าหมายทั้งหมด) · ตอบ Learning Outcome #5 — เรียนต่อเองได้

2. ประเด็นที่ต้องพูด

- แต่ละทีมโชว์ + เล่า: ทำอะไร, ติดอะไร, แก้อย่างไร, ต่อยอดกับงานบริษัทยังไง
- retro: 3 วันทำอะไร, อะไรยาก, อะไร surprising
- เรียนต่อ: docs (derivative.ca) · community (TD forum, Discord) · YouTube (elektronaut, Matthew Ragan) · AI-assisted (Claude/Gemini ช่วยเขียน Script TOP)
- "พวกคุณเริ่มโปรเจกต์เองได้แล้ว — นี่คือเป้าหมาย"

3 + 4. เนื้อหา / OUTPUT

presentation + reflection · ทุกทีมโชว์ installation ที่ทำงาน + รายการ resource · ถ่ายรูป/วิดีโอเก็บ portfolio

RUN OF SHOW

0-5

setup

5-40

โชว์ทีละทีม ~8-10 นาที

40-55

retro รวม

55-60

how-to-continue + ปิด

✂ เตรียม: projector/ลำดับโชว์ · slide resource · ถ่ายรูป/วิดีโอ

Handout: resource เรียนต่อ + ช่อง retro + "โปรเจกต์ต่อไปของฉัน"

Appendix — Master Script Bank · Checklist · Shortcuts

รวม key scripts (illustrative — ปรับตาม version จริง)

```
# Reference/Expression (1.1)
op('lfo1')['chan1'] · op('base1').par.Speed*2 · absTime.seconds · parent().par.Pushforce
# MediaPipe (2.1)
op('handtrack')['index_tip:tx'] - 0.5
# Save + folder watch (2.3)
op('warped').save(f'scans/draw_{int(absTime.seconds)}.png')
# Reset (3.2)
op('feedback1').par.resetpulse.pulse()
# Performance (3.4)
op('render1').cookTime · Perform=F1 · Perf Monitor=Alt+Y · Probe(palette)
```

Pre-workshop install	Shortcut
TouchDesigner (non-commercial)	Tab สร้าง op
webcam + mic	u ขึ้น · i เข้า COMP · h home
mediapipe-touchdesigner plugin	middle-click = info
opencv-contrib-python (ArUco)	F1 Perform · Alt+Y Perf Monitor
(optional) paho-mqtt	scroll=zoom · middle-drag=pan